Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский ядерный университет „МИФИ“»

ОТЧЁТ

по лабораторной работе № 4

по дисциплине «Криптография»

Реализация протокола Диффи–Хеллмана на эллиптических кривых

Вариант № 1 (номер в списке группы — 30)

Выполнил: студент 1-го курса магистратуры
Гафт Илья Маратович

Преподаватель: Епишкина Анна Васильевна

Москва — 2026

1. Цель и постановка задачи

Цель: изучить особенности криптографических протоколов распределения ключей на эллиптических кривых и реализовать учебную модель протокола Диффи–Хеллмана (ECDH).

Задачи: проверить параметры кривой и базовую точку; реализовать модульную арифметику, сложение, удвоение и скалярное умножение точек; получить открытые ключи двух участников; независимо вычислить общий секрет; провести автоматизированное тестирование и проанализировать результаты.

2. Вариант и исходные данные

В таблице 8.1 методических указаний приведено 15 вариантов. Для номера в списке 30: V = (30 mod 15) + 1 = 1.

Параметр	Значение по методичке	Применено
a	−1	−1
b	1	1
p	29	29
G	(9, 27)	(9, 24), обоснование ниже
k_1	4	4
k_2	17	17

Уравнение кривой: $E_{29}(-1,1)$: $y^2 \equiv x^3 - x + 1 \pmod{29}$. Модуль p = 29 — простое число. Проверка невырожденности: $(4a^3 + 27b^2) \bmod p = (-4 + 27) \bmod 29 = 23 \neq 0$. Следовательно, кривая невырождена.

3. Обнаруженная ошибка исходных данных и обоснование корректировки

В методичке для варианта 1 указана базовая точка $G_0 = (9, 27)$. При проверке принадлежности кривой получаем: левая часть $27^2 \bmod 29 = 4$; правая часть $(9^3 - 9 + 1) \bmod 29 = 25$. Поскольку $4 \neq 25$, G_0 не лежит на $E_{29}(-1,1)$. Выполнение групповых операций ECDH с этой точкой математически некорректно.

Чтобы сохранить максимально возможное соответствие варианту, зафиксирована исходная координата x = 9. Решается сравнение $y^2 \equiv 25 \pmod{29}$. Полный перебор $y \in \{0,\ldots,28\}$ даёт y = 5 и y = 24, то есть допустимые точки (9,5) и (9,24). Из них выбрана (9,24), поскольку |24−27|=3 меньше |5−27|=22. Это критерий минимального изменения исходной координаты, а не криптографическое требование.

Повторная проверка: $24^2 \bmod 29 = 25 = (9^3-9+1) \bmod 29$. Порядок выбранной точки равен 37: 37G = O; 37 — простое число, оба секретных скаляра 4 и 17 лежат в диапазоне 1…36. Исправление затрагивает только координату y базовой точки и обусловлено математической некорректностью исходного значения в таблице 8.1.

4. Математическая модель и алгоритм

Нейтральный элемент группы точек — бесконечно удалённая точка O, представленная в программе значением None. Для противоположных точек (x,y) и (x,−y mod p) сумма равна O. Для остальных случаев вычисляется $\lambda = (y_2-y_1)\cdot(x_2-x_1)^{-1} \bmod p$; при удвоении $\lambda = (3x_1^2+a)\cdot(2y_1)^{-1} \bmod p$. Далее $x_3 = \lambda^2-x_1-x_2 \bmod p$ и $y_3 = \lambda(x_1-x_3)-y_1 \bmod p$.

Обратный элемент вычисляется расширенным алгоритмом Евклида. Скалярное умножение kP реализовано методом double-and-add: последовательным анализом битов k, удвоением текущего слагаемого и условным добавлением его к результату. Вычислительная сложность — O(log k) групповых операций.

Протокол: Alice вычисляет $Q_a=k_1G$ и передаёт Bob; Bob вычисляет $Q_\beta=k_2G$ и передаёт Alice. Далее Alice вычисляет $K_a=k_1Q_\beta$, Bob — $K_\beta=k_2Q_a$. По ассоциативности группового закона $K_a=K_\beta=(k_1k_2)G$. Проверяются принадлежность открытых точек кривой, принадлежность подгруппе G и отсутствие O. В демонстрации фиксированные учебные секреты заданы методичкой; в реальном протоколе нужны случайные секреты, аутентификация открытых ключей и KDF для получения симметричного ключа.

5. Организация программы

Файл	Назначение
src/elliptic_curve.py	Проверка кривой и точек, обратный элемент, операции группы, порядок точки, поиск по x
src/ecdh.py	Параметры ECDH, участники, проверка открытых ключей, общий секрет
src/main.py	Демонстрационный сценарий и объяснение исправления точки
tests/test_ecdh.py	27 автоматических тестов unittest

6. Результаты вычислений

Операция	Полученная точка
G	(9, 24)
2G	(12, 8)
3G	(1, 1)
$Q_a = 4G$	(25, 17)
$Q_\beta = 17G$	(11, 4)
$K_a = 4Q_\beta$	(28, 28)
$K_\beta = 17Q_a$	(28, 28)
(4·17)G = 68G	(28, 28)

Получено $K_a = K_\beta = (28,28)$. Контрольное вычисление 68G даёт ту же точку. Таким образом, общий секрет совпадает у обоих участников. Координаты точки общего секрета не следует без дополнительной обработки непосредственно считать готовым симметричным ключом.

7. Тестирование и подтверждающие скриншоты

Проведены проверка синтаксиса Python, запуск демонстрационной программы и автоматизированное тестирование. Команда python3 -m unittest discover -s lab04/tests -v завершилась результатом Ran 27 tests, OK. При запуске python3 lab04/src/main.py получен общий секрет K=(28,28). Результаты выполнения приведены на скриншотах. Тесты охватывают корректные и некорректные параметры, арифметику точек, граничные случаи, обе базовые точки и совпадение общего секрета.



Рисунок 1 — Запуск автоматических тестов: 27 проверок, OK.



Рисунок 2 — Проверка исходной точки и обоснованный поиск допустимой базовой точки.



Рисунок 3 — Открытые ключи, общий секрет и контрольное вычисление.

8. Анализ результатов и ограничения

Проверка дискриминанта подтверждает невырожденность кривой, но сама по себе не подтверждает корректность G; поэтому программа отдельно проверяет принадлежность базовой точки. Отказ от некорректной G_0 и поиск допустимой точки с тем же x позволяют выполнить протокол без изменения a, b, p, k_1, k_2. Изменение базовой точки явно зафиксировано в исходных данных, алгоритме и результатах работы.

Числа p=29 и ord(G)=37 малы и не обеспечивают криптостойкости: секрет можно найти полным перебором. Модель предназначена исключительно для учебной демонстрации. Реальный ECDH требует стандартизованной кривой, криптографически стойкого генератора случайных чисел, аутентификации и KDF; представленная реализация с ветвлениями по секретному скаляру не является защищённой от атак по побочным каналам.

9. Ответы на контрольные вопросы

9.1. Цель применения протокола Диффи–Хеллмана.

Согласование общего секрета двумя сторонами по открытому каналу без непосредственной передачи самого секрета. Сам по себе протокол не аутентифицирует участников.

9.2. Что представляет собой эллиптическая кривая?

В данной работе это множество решений $y^2 \equiv x^3+ax+b \pmod p$ вместе с точкой O, при ненулевом дискриминанте; на множестве задан абелев групповой закон.

9.3. Какие операции определены на эллиптической кривой?

Сложение точек, удвоение как частный случай сложения, нахождение противоположной точки и скалярное умножение повторными сложениями.

9.4. Как выполнить умножение точки на число?

Методом double-and-add: анализировать двоичные разряды скаляра, удваивать текущую точку и при установленном бите прибавлять её к аккумулятору.

9.5. Как вычислить число, обратное по модулю?

Расширенным алгоритмом Евклида: найти u,v, такие что au+mv=НОД(a,m). При НОД=1 число u mod m является обратным.

9.6. Что является нулём эллиптической кривой?

Бесконечно удалённая точка O, нейтральный элемент: P+O=P.

9.7. Почему эллиптические кривые позволяют уменьшить размер ключей?

Для правильно выбранных групп известные классические атаки на дискретный логарифм эллиптической кривой требуют значительно больших затрат при меньших размерах параметров, чем в ряде классических систем сопоставимой стойкости.

9.8. Как выглядит общее уравнение?

Для простого поля характеристики больше 3: $y^2 \equiv x^3+ax+b \pmod p$, при $4a^3+27b^2 \neq 0 \pmod p$.

9.9. Какие данные передаются участниками?

Открытые параметры кривой и базовая точка согласованы заранее; участники обмениваются открытыми точками $Q_a=k_1G$ и $Q_\beta=k_2G$. Секретные k_1,k_2 не передаются.

9.10. Почему перехват открытых данных не раскрывает общий секрет?

При криптографически стойких параметрах восстановление секретного скаляра из G и kG вычислительно трудно (задача ECDLP). Для маленького учебного p=29 эта защита отсутствует.

9.11. Какой объект используется как общий секрет?

Точка эллиптической кривой $K=(k_1k_2)G$; для применения в симметричной криптографии из согласованного секрета получают ключ с помощью KDF.

9.12. Какие исходные данные необходимы?

Параметры a,b,p, базовая точка G (и её порядок n для проверки диапазонов), секретные скаляры участников k_1 и k_2.

9.13. Как проверить корректность выбранной кривой?

Проверить простоту p, p>3, ненулевой дискриминант; отдельно проверить координаты G по уравнению и порядок точки.

9.14. Опишите алгоритм умножения точки на число.

Инициализировать R=O, A=P; пока k>0, при нечётном k выполнить R=R+A; затем A=2A и k=k//2; вернуть R.

9.15. Какие атаки возможны на ECDH?

MITM без аутентификации, атаки на некорректные точки и малые подгруппы при отсутствии проверок, перебор/дискретное логарифмирование на слабых кривых, атаки по времени и иным побочным каналам.

10. Выводы

Реализована учебная модель ECDH на эллиптической кривой над F_{29}. Обнаружена и строго обоснована ошибка базовой точки в исходных данных варианта № 1; для демонстрации выбрана допустимая точка (9,24) с сохранением координаты x. Проверены операции сложения, удвоения и скалярного умножения, получены открытые ключи и общий секрет (28,28). Все 27 автоматических тестов завершились успешно. Ограничения учебной модели и необходимость согласования замены исходной точки зафиксированы.

Приложение А. Полный исходный код

src/elliptic_curve.py

```
"""
Лабораторная работа № 4.
Арифметика точек эллиптической кривой над конечным полем F_p.

Кривая:
    y^2 = x^3 + ax + b (mod p)

Точка на бесконечности представляется значением None.

Реализация предназначена для учебных целей.
"""

from dataclasses import dataclass
from typing import Optional, Tuple


Point = Optional[Tuple[int, int]]


def is_prime(n: int) -> bool:
    """Проверка простоты числа методом пробного деления."""
    if n < 2:
        return False

    if n == 2:
        return True

    if n % 2 == 0:
        return False

    divisor = 3

    while divisor * divisor <= n:
        if n % divisor == 0:
            return False

        divisor += 2

    return True


def mod_inverse(value: int, modulus: int) -> int:
    """
    Вычисление обратного элемента по модулю
    расширенным алгоритмом Евклида.

    Возвращает x:
        value * x = 1 (mod modulus)
    """
    if modulus <= 1:
        raise ValueError("Модуль должен быть больше 1")

    value %= modulus

    if value == 0:
        raise ValueError(
            "Обратный элемент для нуля не существует"
```

```
        )

    old_r, r = modulus, value
    old_t, t = 0, 1

    while r != 0:
        quotient = old_r // r

        old_r, r = r, old_r - quotient * r
        old_t, t = t, old_t - quotient * t

    if old_r != 1:
        raise ValueError(
            "Обратный элемент не существует"
        )

    return old_t % modulus


def find_points_by_x(curve, x: int) -> list:
    """
    Поиск всех точек эллиптической кривой
    с заданной координатой x.

    Используется перебор значений y от 0 до p-1.
    Подходит для учебных кривых малого размера.
    """
    if not 0 <= x < curve.p:
        raise ValueError(
            "Координата x должна принадлежать полю F_p"
        )

    return [
        (x, y)
        for y in range(curve.p)
        if curve.is_on_curve((x, y))
    ]


@dataclass(frozen=True)
class EllipticCurve:
    """Эллиптическая кривая над простым полем F_p."""

    a: int
    b: int
    p: int

    def __post_init__(self):
        if not is_prime(self.p):
            raise ValueError(
                "Модуль p должен быть простым числом"
            )

        if self.p <= 3:
            raise ValueError(
                "Для данной реализации требуется p > 3"
            )

        discriminant = (
            4 * self.a**3 + 27 * self.b**2
        ) % self.p

        if discriminant == 0:
            raise ValueError(
                "Эллиптическая кривая вырождена"
            )

    def discriminant(self) -> int:
        """Дискриминант кривой по модулю p."""
        return (
            4 * self.a**3 + 27 * self.b**2
        ) % self.p

    def is_on_curve(self, point: Point) -> bool:
        """Проверка принадлежности точки кривой."""
        if point is None:
            return True

        x, y = point
```

```
    if not (0 <= x < self.p and 0 <= y < self.p):
        return False

    left = y**2 % self.p

    right = (
        x**3 + self.a * x + self.b
    ) % self.p

    return left == right

def validate_point(self, point: Point) -> None:
    """Проверка корректности точки."""
    if not self.is_on_curve(point):
        raise ValueError(
            f"Точка {point} не принадлежит кривой"
        )

def negate(self, point: Point) -> Point:
    """Получение противоположной точки."""
    self.validate_point(point)

    if point is None:
        return None

    x, y = point

    return x, (-y) % self.p

def add(self, p1: Point, p2: Point) -> Point:
    """Сложение двух точек эллиптической кривой."""
    self.validate_point(p1)
    self.validate_point(p2)

    # P + O = P
    if p1 is None:
        return p2

    # O + P = P
    if p2 is None:
        return p1

    x1, y1 = p1
    x2, y2 = p2

    # P + (-P) = O
    if x1 == x2 and (y1 + y2) % self.p == 0:
        return None

    if p1 == p2:
        # Удвоение точки
        numerator = (
            3 * x1**2 + self.a
        ) % self.p

        denominator = (
            2 * y1
        ) % self.p

    else:
        # Сложение различных точек
        numerator = (
            y2 - y1
        ) % self.p

        denominator = (
            x2 - x1
        ) % self.p

    slope = (
        numerator
        * mod_inverse(denominator, self.p)
    ) % self.p

    x3 = (
        slope**2 - x1 - x2
    ) % self.p
```

```
        y3 = (
            slope * (x1 - x3) - y1
        ) % self.p

        result = (x3, y3)

        self.validate_point(result)

        return result

    def double(self, point: Point) -> Point:
        """Удвоение точки."""
        return self.add(point, point)

    def multiply(self, k: int, point: Point) -> Point:
        """
        Умножение точки на скаляр методом double-and-add.

        Сложность: O(log k).
        """
        self.validate_point(point)

        if not isinstance(k, int):
            raise TypeError(
                "Скаляр должен быть целым числом"
            )

        if k == 0 or point is None:
            return None

        if k < 0:
            return self.multiply(
                -k,
                self.negate(point)
            )

        result = None
        addend = point

        while k > 0:
            if k & 1:
                result = self.add(result, addend)

            addend = self.double(addend)

            k >>= 1

        return result

    def point_order(self, point: Point) -> int:
        """
        Вычисление порядка точки перебором.

        Используется только для учебных
        кривых малого размера.
        """
        self.validate_point(point)

        if point is None:
            return 1

        result = None

        # Верхняя граница по теореме Хассе
        limit = self.p + 1 + 2 * (
            self.p ** 0.5
        )

        for k in range(1, int(limit) + 2):
            result = self.add(result, point)

            if result is None:
                return k

        raise RuntimeError(
            "Не удалось определить порядок точки"
        )
```

src/ecdh.py

```
"""
Лабораторная работа № 4.
Протокол Диффи–Хеллмана на эллиптических кривых.
"""

from dataclasses import dataclass

from elliptic_curve import EllipticCurve, Point


@dataclass(frozen=True)
class ECDHParameters:
    """Параметры протокола ECDH."""

    curve: EllipticCurve
    generator: Point

    def __post_init__(self):
        if self.generator is None:
            raise ValueError(
                "Базовая точка не может быть O"
            )

        self.curve.validate_point(self.generator)

        if self.curve.point_order(self.generator) <= 2:
            raise ValueError(
                "Порядок базовой точки слишком мал"
            )


class ECDHParticipant:
    """Участник протокола ECDH."""

    def __init__(
        self,
        name: str,
        parameters: ECDHParameters,
        private_key: int
    ):
        self.name = name
        self.parameters = parameters

        order = parameters.curve.point_order(
            parameters.generator
        )

        if not isinstance(private_key, int):
            raise TypeError(
                "Закрытый ключ должен быть целым числом"
            )

        if not 1 <= private_key < order:
            raise ValueError(
                f"Закрытый ключ должен быть "
                f"в диапазоне [1, {order - 1}]"
            )

        self.private_key = private_key

        self.public_key = (
            parameters.curve.multiply(
                private_key,
                parameters.generator
            )
        )

        if self.public_key is None:
            raise ValueError(
                "Открытый ключ не может быть O"
            )

    def compute_shared_secret(
        self,
        other_public_key: Point
    ) -> Point:
        """Вычисление общего секрета."""
```

```
        curve = self.parameters.curve

        curve.validate_point(other_public_key)

        if other_public_key is None:
            raise ValueError(
                "Получен недопустимый открытый ключ O"
            )

        order = curve.point_order(
            self.parameters.generator
        )

        if curve.multiply(
            order,
            other_public_key
        ) is not None:
            raise ValueError(
                "Открытый ключ не принадлежит "
                "подгруппе базовой точки"
            )

        shared_secret = curve.multiply(
            self.private_key,
            other_public_key
        )

        if shared_secret is None:
            raise ValueError(
                "Общий секрет не может быть O"
            )

        return shared_secret


def perform_ecdh(
    parameters: ECDHParameters,
    k1: int,
    k2: int
) -> dict:
    """Моделирование обмена ключами."""

    alice = ECDHParticipant(
        "Alice",
        parameters,
        k1
    )

    bob = ECDHParticipant(
        "Bob",
        parameters,
        k2
    )

    # Обмен открытыми ключами
    public_a = alice.public_key
    public_b = bob.public_key

    # Независимое вычисление секрета
    shared_a = alice.compute_shared_secret(
        public_b
    )

    shared_b = bob.compute_shared_secret(
        public_a
    )

    return {
        "public_a": public_a,
        "public_b": public_b,
        "shared_a": shared_a,
        "shared_b": shared_b,
        "keys_match": shared_a == shared_b,
    }
```

src/main.py

```
"""
```

```
НИЯУ МИФИ
Лабораторная работа № 4

Тема:
Реализация протокола Диффи–Хеллмана
на эллиптических кривых.

Студент: Гафт Илья Маратович
Номер в списке: 30
Вариант: 1
"""

from elliptic_curve import (
    EllipticCurve,
    find_points_by_x,
)
from ecdh import ECDHParameters, perform_ecdh


def print_separator():
    print("=" * 65)


def main():
    print_separator()
    print("ЛАБОРАТОРНАЯ РАБОТА № 4")
    print("Протокол Диффи–Хеллмана на эллиптических кривых")
    print("Вариант № 1")
    print_separator()

    # Исходные параметры варианта № 1
    a = -1
    b = 1
    p = 29

    original_g = (9, 27)

    k1 = 4
    k2 = 17

    print("\n1. ИСХОДНЫЕ ДАННЫЕ")

    print(f"a = {a}")
    print(f"b = {b}")
    print(f"p = {p}")
    print(f"G из методички = {original_g}")
    print(f"k1 = {k1}")
    print(f"k2 = {k2}")

    # Создание и проверка кривой
    curve = EllipticCurve(a, b, p)

    print("\n2. ПРОВЕРКА КРИВОЙ")

    print(
        "Уравнение: "
        "y^2 = x^3 - x + 1 (mod 29)"
    )

    print(
        "Дискриминант:",
        curve.discriminant()
    )

    print("Кривая корректна: ДА")

    # Проверка точки из методички
    print("\n3. ПРОВЕРКА БАЗОВОЙ ТОЧКИ")

    x, y = original_g

    left = y ** 2 % p
    right = (x ** 3 + a * x + b) % p

    print(f"Исходная точка G = {original_g}")

    print("\nПроверка уравнения:")
    print(f"y² mod {p} = {left}")
    print(f"(x³ + ax + b) mod {p} = {right}")
```

```
if curve.is_on_curve(original_g):
    print("\nТочка принадлежит кривой: ДА")
    generator = original_g

else:
    print(f"\n{left} != {right}")
    print("Точка принадлежит кривой: НЕТ")

    print("\nПОИСК КОРРЕКТНОЙ БАЗОВОЙ ТОЧКИ")

    print(f"Сохраняем координату x = {x}")

    candidates = find_points_by_x(curve, x)

    print(
        f"\ny² = ({x}³ + ({a})*{x} + {b}) mod {p}"
    )
    print(f"y² = {right} (mod {p})")

    print("\nНайденные точки:")

    for point in candidates:
        print(f"G = {point}")

    if not candidates:
        raise ValueError(
            "Не найдены точки с заданной координатой x"
        )

    # Выбираем допустимую точку,
    # максимально близкую к исходной по y.
    generator = min(
        candidates,
        key=lambda point: (
            abs(point[1] - y),
            point[1]
        )
    )

    print(
        "\nКритерий выбора: минимальное "
        "изменение координаты y"
    )

    print(f"Выбрана точка G = {generator}")

    gx, gy = generator

    new_left = gy ** 2 % p

    new_right = (
                    gx ** 3 + a * gx + b
                ) % p

    print("\nПовторная проверка:")
    print(f"{gy}² mod {p} = {new_left}")
    print(
        f"({gx}³ + ({a})*{gx} + {b}) "
        f"mod {p} = {new_right}"
    )

    assert new_left == new_right

    print("\nТочка принадлежит кривой: ДА")

# Проверка выбранной точки
curve.validate_point(generator)

parameters = ECDHParameters(
    curve,
    generator
)

order = curve.point_order(generator)

print("\n4. ПАРАМЕТРЫ ПРОТОКОЛА")

print(f"G = {generator}")
```

```
    print(f"Порядок точки G = {order}")

    # Демонстрация операций
    print("\n5. ОПЕРАЦИИ НАД ТОЧКАМИ")

    double_g = curve.double(generator)

    triple_g = curve.add(
        double_g,
        generator
    )

    print(f"2G = {double_g}")
    print(f"3G = {triple_g}")

    # Выполнение ECDH
    print("\n6. ОБМЕН ОТКРЫТЫМИ КЛЮЧАМИ")

    result = perform_ecdh(
        parameters,
        k1,
        k2
    )

    print(
        f"Alice: QA = {k1}G = "
        f"{result['public_a']}"
    )

    print(
        f"Bob:   QB = {k2}G = "
        f"{result['public_b']}"
    )

    print("\n7. ВЫЧИСЛЕНИЕ ОБЩЕГО СЕКРЕТА")

    print(
        f"Alice: K = {k1} * QB = "
        f"{result['shared_a']}"
    )

    print(
        f"Bob:   K = {k2} * QA = "
        f"{result['shared_b']}"
    )

    print("\n8. ПРОВЕРКА СОВПАДЕНИЯ")

    print(
        "Общие секреты совпадают:",
        result["keys_match"]
    )

    assert result["keys_match"], (
        "Ошибка: общие секреты не совпадают"
    )

    # Независимая математическая проверка
    expected = curve.multiply(
        k1 * k2,
        generator
    )

    print(
        "Контрольное вычисление "
        "(k1*k2)*G:",
        expected
    )

    assert expected == result["shared_a"]

    print("\nРЕЗУЛЬТАТ: ПРОТОКОЛ ВЫПОЛНЕН УСПЕШНО")

    print_separator()


if __name__ == "__main__":
    main()
```

tests/test_ecdh.py

```
"""
Автоматические тесты лабораторной работы № 4.

Запуск:
python3 -m unittest discover -s lab04/tests -v
"""

import sys
import unittest
from pathlib import Path


SRC_DIR = (
    Path(__file__).resolve().parents[1] / "src"
)

sys.path.insert(0, str(SRC_DIR))


from elliptic_curve import (
    EllipticCurve,
    is_prime,
    mod_inverse,
    find_points_by_x,
)

from ecdh import (
    ECDHParameters,
    ECDHParticipant,
    perform_ecdh,
)


class TestEllipticCurve(unittest.TestCase):

    def setUp(self):
        self.curve = EllipticCurve(
            a=-1,
            b=1,
            p=29
        )

        self.g = (0, 1)

    def test_01_prime_modulus(self):
        self.assertTrue(is_prime(29))
        self.assertFalse(is_prime(21))
        self.assertFalse(is_prime(1))

    def test_02_discriminant(self):
        self.assertEqual(
            self.curve.discriminant(),
            23
        )

    def test_03_invalid_curve(self):
        with self.assertRaises(ValueError):
            EllipticCurve(
                a=0,
                b=0,
                p=29
            )

    def test_04_composite_modulus(self):
        with self.assertRaises(ValueError):
            EllipticCurve(
                a=-1,
                b=1,
                p=21
            )

    def test_05_generator_on_curve(self):
        self.assertTrue(
            self.curve.is_on_curve(self.g)
        )

    def test_06_original_generator_invalid(self):
```

```
    self.assertFalse(
        self.curve.is_on_curve((9, 27))
    )

def test_07_invalid_point_rejected(self):
    with self.assertRaises(ValueError):
        self.curve.validate_point((9, 27))

def test_08_mod_inverse(self):
    self.assertEqual(
        mod_inverse(7, 29),
        25
    )

def test_09_mod_inverse_zero(self):
    with self.assertRaises(ValueError):
        mod_inverse(0, 29)

def test_10_point_at_infinity(self):
    self.assertEqual(
        self.curve.add(self.g, None),
        self.g
    )

    self.assertEqual(
        self.curve.add(None, self.g),
        self.g
    )

def test_11_opposite_points(self):
    negative = self.curve.negate(self.g)

    self.assertIsNone(
        self.curve.add(self.g, negative)
    )

def test_12_point_doubling(self):
    self.assertEqual(
        self.curve.double(self.g),
        (22, 10)
    )

def test_13_point_addition(self):
    two_g = self.curve.double(self.g)

    three_g = self.curve.add(
        two_g,
        self.g
    )

    self.assertEqual(
        three_g,
        (27, 13)
    )

def test_14_scalar_multiplication(self):
    self.assertEqual(
        self.curve.multiply(4, self.g),
        (9, 24)
    )

def test_15_zero_scalar(self):
    self.assertIsNone(
        self.curve.multiply(0, self.g)
    )

def test_16_negative_scalar(self):
    self.assertEqual(
        self.curve.multiply(-1, self.g),
        self.curve.negate(self.g)
    )

def test_17_point_order(self):
    self.assertEqual(
        self.curve.point_order(self.g),
        37
    )

def test_18_ecdh_public_keys(self):
```

```
    params = ECDHParameters(
        self.curve,
        self.g
    )

    result = perform_ecdh(
        params,
        4,
        17
    )

    self.assertEqual(
        result["public_a"],
        (9, 24)
    )

    self.assertEqual(
        result["public_b"],
        (20, 21)
    )

def test_19_ecdh_shared_secret(self):
    params = ECDHParameters(
        self.curve,
        self.g
    )

    result = perform_ecdh(
        params,
        4,
        17
    )

    self.assertEqual(
        result["shared_a"],
        (11, 4)
    )

    self.assertEqual(
        result["shared_b"],
        (11, 4)
    )

    self.assertTrue(
        result["keys_match"]
    )

def test_20_independent_verification(self):
    params = ECDHParameters(
        self.curve,
        self.g
    )

    result = perform_ecdh(
        params,
        4,
        17
    )

    expected = self.curve.multiply(
        4 * 17,
        self.g
    )

    self.assertEqual(
        result["shared_a"],
        expected
    )

def test_21_invalid_private_key(self):
    params = ECDHParameters(
        self.curve,
        self.g
    )

    with self.assertRaises(ValueError):
        ECDHParticipant(
            "Alice",
            params,
```

```
            0
        )

    def test_22_invalid_public_key(self):
        params = ECDHParameters(
            self.curve,
            self.g
        )

        alice = ECDHParticipant(
            "Alice",
            params,
            4
        )

        with self.assertRaises(ValueError):
            alice.compute_shared_secret(
                (9, 27)
            )

    def test_23_infinity_public_key(self):
        params = ECDHParameters(
            self.curve,
            self.g
        )

        alice = ECDHParticipant(
            "Alice",
            params,
            4
        )

        with self.assertRaises(ValueError):
            alice.compute_shared_secret(None)


    def test_24_corrected_generator(self):
        """Проверка исправленной точки варианта № 1."""
        generator = (9, 24)

        self.assertTrue(
            self.curve.is_on_curve(generator)
        )

        self.assertEqual(
            self.curve.point_order(generator),
            37
        )

    def test_25_corrected_generator_public_keys(self):
        """Проверка открытых ключей варианта № 1."""
        params = ECDHParameters(
            self.curve,
            (9, 24)
        )

        result = perform_ecdh(params, 4, 17)

        self.assertEqual(
            result["public_a"],
            (25, 17)
        )

        self.assertEqual(
            result["public_b"],
            (11, 4)
        )

    def test_26_corrected_generator_shared_secret(self):
        """Проверка общего секрета варианта № 1."""
        params = ECDHParameters(
            self.curve,
            (9, 24)
        )

        result = perform_ecdh(params, 4, 17)

        self.assertEqual(
            result["shared_a"],
```

```
            (28, 28)
        )

        self.assertEqual(
            result["shared_b"],
            (28, 28)
        )

        self.assertTrue(result["keys_match"])

        self.assertEqual(
            result["shared_a"],
            self.curve.multiply(68, (9, 24))
        )

    def test_27_find_points_by_x(self):
        """Поиск допустимых точек с координатой x=9."""

        points = find_points_by_x(
            self.curve,
            9
        )

        self.assertEqual(
            points,
            [(9, 5), (9, 24)]
        )

        original_y = 27

        selected = min(
            points,
            key=lambda point: (
                abs(point[1] - original_y),
                point[1]
            )
        )

        self.assertEqual(
            selected,
            (9, 24)
        )


if __name__ == "__main__":
    unittest.main()
```

Приложение Б. Команды запуска

```
python3 -m compileall -q lab04/src lab04/tests
python3 -m unittest discover -s lab04/tests -v
python3 lab04/src/main.py
```